

Vim: To'liq va Mukammal Darslik

Docker konteynerlari, remote server'lar va boshqa "GUI'siz" muhitlarda ishlash uchun eng muhim ko'nikma — bu Vim'ni bilish. Bu darslik sizni noldan boshlab professional darajagacha olib boradi.

Mundarija

1. Vim nima va nega u kerak
 2. Modal Editing tushunchasi
 3. Ishga tushirish va chiqish
 4. Harakatlanish (Motions)
 5. Insert rejimi
 6. Operator + Motion grammatikasi
 7. O'chirish, nusxalash, joylashtirish
 8. Undo, Redo va takrorlash
 9. Visual rejim
 10. Qidirish va almashtirish
 11. Registers (Bufer registrarlari)
 12. Marks va Jumps
 13. Bir nechta fayl bilan ishlash: Buffers, Windows, Tabs
 14. Macros (Makroslar)
 15. .vimrc sozlash
 16. Pluginlar
 17. Docker va server muhitida Vim
 18. Cheat Sheet
-

1. Vim nima va nega u kerak

Vim (Vi IMproved) — 1991-yilda Bram Moolenaar tomonidan yaratilgan, klassik `(vi)` editorining kengaytirilgan versiyasi. U deyarli har qanday Unix/Linux tizimida oldindan o'rnatilgan (yoki `(vi)` sifatida mavjud) bo'ladi.

Nega backend/Docker ishlarida Vim kerak:

- Docker konteynerida odatda faqat minimal shell mavjud — GUI editor yo'q, ba'zida hatto `(nano)` ham yo'q.
- SSH orqali serverga ulanganingizda, faylni tahrirlash uchun terminal ichida ishlaydigan editor kerak.

- `git commit`, `crontab -e`, `visudo` kabi ko'plab tizim buyruqlari default holda Vim/Vi'ni ochadi.
- Vim **modal** bo'lgani uchun, o'rgangandan keyin sichqonchasiz, klaviaturadan qo'l uzmasdan juda tez tahrirlash imkonini beradi.

Vim vs VSCode farqi: VSCode — bu "modeless" editor: doim yozish rejimida bo'lasiz, buyruqlar uchun menyu yoki qisqa tugmalar ishlatasiz. Vim esa **rejimlarga (modes)** asoslangan — bu uning kuchi va birinchi qiyinchiligi.

2. Modal Editing tushunchasi

Vim'ning eng muhim tushunchasi — u bir nechta **rejimda** ishlaydi. Har bir rejimda klaviatura tugmalari boshqacha ma'noga ega bo'ladi.

Rejim	Vazifasi	Qanday kirish
Normal	Harakatlanish va buyruq berish (default rejim)	<code>[Esc]</code>
Insert	Matn kiritish (odatiy yozish)	<code>[i]</code> , <code>[a]</code> , <code>[o]</code> va h.k.
Visual	Matnni belgilash (tanlash)	<code>[v]</code> , <code>[V]</code> , <code>[Ctrl+v]</code>
Command-line	<code>[:]</code> bilan boshlanadigan buyruqlar	<code>[:]</code>
Replace	Mavjud matnni ustidan yozish	<code>[R]</code>

Muhim qoida: Vim ochilganda siz doim **Normal rejimda** bo'lasiz. Bu yerda harflar buyruq sifatida ishlaydi (masalan `[x]` — belgini o'chiradi), matn yozish uchun emas.

Normal rejim ↔ Insert rejim

↑ Esc bosilganda Normal'ga qaytiladi

↓ i, a, o, I, A, O bosilganda Insert'ga kiriladi

Yangi boshlovchilarning eng katta xatosi — Insert rejimda bo'lib, Normal rejim buyruqlarini yozishga urinish (masalan matn ichiga tasodifan "dd" yozib qo'yish). Har doim **qaysi rejimda ekanligingizni bilib turing** (pastki statusbar'da odatda `-- INSERT --` deb yoziladi).

3. Ishga tushirish va chiqish

bash

```
vim fayl.txt          # faylni ochish (mavjud bo'lmasa, yangi yaratiladi)
vim +10 fayl.txt      # 10-qatordan ochish
vim +/qidiruv fayl.txt # "qidiruv" so'ziga o'tib ochish
```

Chiqish buyruqlari (Command-line rejimda, ya'ni `:` bosgandan keyin)

Buyruq	Vazifasi
<code>:w</code>	saqlash (write)
<code>:q</code>	chiqish (agar o'zgarish bo'lmasa)
<code>:wq</code> yoki <code>:x</code>	saqlab chiqish
<code>:q!</code>	saqlamasdan majburiy chiqish
<code>:wq!</code>	majburiy saqlab chiqish
<code>ZZ</code>	(Normal rejimda) saqlab chiqish — <code>:wq</code> ning tezkor varianti
<code>:w yangi_nom.txt</code>	boshqa nom bilan saqlash

Eslatma: Agar tasodifan Vim'ga tushib qolib, chiqishni bilmasangiz — `(Esc)` bosing, so'ng `:q!` yozib `(Enter)` bosing.

4. Harakatlanish (Motions)

Vim'da sichqoncha ishlatilmaydi — barcha harakatlanish klaviaturadan amalga oshiriladi. Bu tezlikning asosiy sababi.

Asosiy yo'nalishlar

```
k (yuqoriga)
h (chap)  l (o'ng)
j (pastga)
```

`(h j k l)` — o'ng qo'lning asosiy joylashuvida bo'lgani uchun tanlangan, strelka tugmalaridan tezroq.

So'z bo'yicha harakatlanish

Buyruq	Vazifasi
<code>(w)</code>	keyingi so'z boshiga o'tish

Buyruq	Vazifasi
b	oldingi so'z boshiga o'tish
e	joriy/keyingi so'z oxiriga o'tish
W , B , E	xuddi shu, lekin bo'shliq bilan ajratilgan "katta so'zlar" (punktuatsiyani hisobga olmaydi)

Qator ichida harakatlanish

Buyruq	Vazifasi
0	qator boshiga (birinchi belgi)
^	qatordagi birinchi bo'sh bo'lmagan belgiga
\$	qator oxiriga
f<belgi>	shu qatorda <belgi> ga o'tish (forward find)
t<belgi>	<belgi> dan oldingi belgiga o'tish (till)
F<belgi> , T<belgi>	xuddi shu, lekin orqaga qarab
;	oxirgi f / t qidiruvini takrorlash
,	oxirgi f / t qidiruvini teskari yo'nalishda takrorlash

Fayl ichida harakatlanish

Buyruq	Vazifasi
gg	faylning birinchi qatoriga
G	faylning oxirgi qatoriga
:{raqam} yoki {raqam}G	ma'lum qatorga o'tish (masalan :42)
Ctrl+d	yarim ekran pastga
Ctrl+u	yarim ekran yuqoriga
Ctrl+f	bir ekran pastga (forward)
Ctrl+b	bir ekran yuqoriga (back)
%	mos qavsga o'tish ((, { , [)

Buyruq	Vazifasi
{	oldingi bo'sh qatorga (paragraf)
}	keyingi bo'sh qatorga (paragraf)

Sonlar bilan birga ishlatish

Har qanday harakatni son bilan birlashtirish mumkin — bu Vim'ning kuchli tomonlaridan biri:

```
5j    → 5 qator pastga
3w    → 3 ta so'z oldinga
10G   → 10-qatorga o'tish
```

5. Insert rejimi

Matn kiritish uchun Normal rejimdan Insert rejimga o'tish kerak. Buning bir necha yo'li bor va har biri kursorni boshqacha joyga qo'yadi:

Buyruq	Vazifasi
i	kursor turgan joyning oldiga (insert)
a	kursor turgan joyning keyiniga (append)
I	qator boshiga o'tib, insert
A	qator oxiriga o'tib, insert
O	pastda yangi qator ochib, insert
O	yuqorida yangi qator ochib, insert
Esc	Insert rejimdan chiqib, Normal rejimga qaytish

Maslahat: `Esc` o'rniga `Ctrl+[` ham ishlaydi va qo'lni asosiy klaviaturadan uzoqlashtirmaydi.

6. Operator + Motion grammatikasi

Bu — Vim'ning **eng muhim va eng kuchli** g'oyasi. Vim buyruqlari til grammatikasiga o'xshaydi:

OPERATOR + MOTION = BUYRUQ

Ya'ni, "nima qilish" (operator) va "qayerda" (motion) birlashadi.

Asosiy operatorlar

Operator	Vazifasi
d	delete (o'chirish)
c	change (o'chirib, Insert rejimga o'tish)
y	yank (nusxalash / copy)
>	right indent qilish
<	left indent qilish
=	avtomatik formatlash

Misollar

```
dw  → kursordan so'z oxirigacha o'chirish (delete word)
d$  → kursordan qator oxirigacha o'chirish
d0  → kursordan qator boshigacha o'chirish
dG  → joriy qatordan fayl oxirigacha o'chirish
dgg → joriy qatordan fayl boshigacha o'chirish
cw  → so'zni o'chirib, o'rniga yozish
ciw → kursor turgan so'zni to'liq o'chirib, yozish (change inner word)
yy  → butun qatorni nusxalash
y$  → kursordan qator oxirigacha nusxalash
```

Text Objects (matn obyektlari)

Operatorlar bilan birga ishlaydigan maxsus "motion"lar — mantiqiy bloklarni tanlaydi:

Belgi	Vazifasi
<code>iw</code>	inner word — so'zning o'zi
<code>aw</code>	a word — so'z + atrofidagi bo'shliq
<code>i"</code>	qo'shtirnoq ichidagi matn
<code>a"</code>	qo'shtirnoqlar bilan birga
<code>i()</code> yoki <code>ib</code>	qavs ichidagi matn
<code>i{</code> yoki <code>iB</code>	figurali qavs ichidagi matn
<code>it</code>	HTML/XML tag ichidagi matn
<code>ip</code>	inner paragraph

```
ci"    → qo'shtirnoq ichidagi matnni o'zgartirish: "eski" → "yangi"
di(    → qavs ichidagi hammasini o'chirish: foo(a, b) → foo()
ya{    → figurali qavs bilan birga nusxalash
```

Bu grammatika tufayli, siz kod ichida `printf("...")` kabi joylarda `ci"` bosib, darhol matnni yangilashingiz mumkin — bu VSCode'da bir necha marta bosish bilan qilinadigan ishni bitta buyruqqa aylantiradi.

Buyruqni takrorlash — son bilan

```
2dw    → 2 ta so'zni o'chirish
3dd     → 3 qatorni o'chirish
5cc     → 5 qatorni o'zgartirish
```

7. O'chirish, nusxalash, joylashtirish

Tezkor (bir belgili) buyruqlar

Buyruq	Vazifasi
<code>x</code>	kursor ostidagi belgini o'chirish
<code>X</code>	kursordan oldingi belgini o'chirish
<code>dd</code>	butun qatorni o'chirish

Buyruq	Vazifasi
D	kursordan qator oxirigacha o'chirish (d\$) bilan bir xil)
yy yoki Y	butun qatorni nusxalash
p	o'chirilgan/nusxalangan matnni kursordan keyin joylashtirish
P	kursordan oldin joylashtirish
r<belgi>	bitta belgini almashtirish (replace)
~	katta/kichik harfni almashtirish

Muhim tushuncha: har qanday o'chirish — "nusxalash" ham hisoblanadi

Vim'da alohida "clipboard" tushunchasi yo'q — **d** yoki **x** bilan o'chirilgan matn avtomatik ravishda "unnamed register"ga tushadi va uni **p** bilan joylashtirish mumkin. Bu clipboard'ni "Registers" bo'limida batafsil ko'rib chiqamiz.

8. Undo, Redo va takrorlash

Buyruq	Vazifasi
u	undo (bekor qilish)
Ctrl+r	redo (qayta tiklash)
.	oxirgi o'zgartiruvchi buyruqni takrorlash
U	butun qatordagi o'zgarishlarni bekor qilish

. (nuqta) buyrug'i juda kuchli — masalan **ciw** bilan bitta so'zni o'zgartirgandan keyin, boshqa so'zga o'tib **.** bossangiz, xuddi shu o'zgarish takrorlanadi. Bu bir xil o'zgartirishni bir necha joyda tez qo'llash imkonini beradi.

9. Visual rejim

Visual rejim — matnni "ko'zga ko'rinadigan" tarzda belgilash uchun ishlatiladi (VSCode'dagi mouse bilan tanlashga o'xshaydi, lekin klaviaturadan).

Buyruq	Vazifasi
<code>v</code>	Visual rejim (belgi-belgi tanlash)
<code>V</code>	Visual Line rejim (butun qatorlarni tanlash)
<code>Ctrl+v</code>	Visual Block rejim (to'g'ri burchakli blok tanlash — juda foydali!)
<code>Esc</code>	Visual rejimdan chiqish

Visual rejimda matn tanlab, so'ng operator qo'llash mumkin:

```
v + motion + d    → tanlangan matnni o'chirish
v + motion + y    → tanlangan matnni nusxalash
v + motion + >    → tanlangan matnni indent qilish
```

Visual Block — masalan bir nechta qatorning boshiga bir vaqtda `//` qo'shish uchun ishlatiladi:

```
1. Ctrl+v bosing
2. j bilan bir nechta qatorni pastga tanlang
3. I bosing, // yozing
4. Esc bosing → barcha tanlangan qatorlarga bir vaqtda qo'shiladi
```

10. Qidirish va almashtirish

Qidirish

Buyruq	Vazifasi
<code>/qidiruv</code>	oldinga qarab qidirish
<code>?qidiruv</code>	orqaga qarab qidirish
<code>n</code>	keyingi moslikka o'tish
<code>N</code>	oldingi moslikka (teskari yo'nalishda)
<code>*</code>	kursor ostidagi so'zni oldinga qidirish
<code>#</code>	kursor ostidagi so'zni orqaga qidirish

Almashtirish (Substitute) — `:s` buyrug'i

```
:s/eski/yangi/      → joriy qatordagi birinchi "eski"ni "yangi"ga
almashtiradi
:s/eski/yangi/g      → joriy qatordagi barcha "eski"larni almashtiradi
:%s/eski/yangi/g      → butun fayldagi barcha "eski"larni almashtiradi
:%s/eski/yangi/gc     → har birini tasdiqlab almashtirish (confirm)
:s,10s/eski/yangi/g  → 5-10 qatorlar oralig'ida almashtirish
:%s/\<so'z\>/yangi/g → faqat to'liq mos keladigan so'zni almashtirish
```

Bu yerda **%** — "butun fayl" degani, **g** — "global" (qatordagi barcha mosliklar), **c** — "confirm" (har birini so'rab tasdiqlash).

Regex (regular expressions) ham to'liq qo'llab-quvvatlanadi, bu Vim'ni matn bilan ishlashda juda kuchli qiladi.

11. Registers (Bufer registrlari)

Vim'da 26 tagacha nomlangan register mavjud (**a** dan **z** gacha) — bu "bir nechta clipboard"ga o'xshaydi.

```
"ayy      → joriy qatorni "a" registriga nusxalash
"ap       → "a" registridagi matnni joylashtirish
"add      → qatorni o'chirib, "a" registriga saqlash
```

Foydali maxsus registrlar:

Register	Mazmuni
"0	oxirgi yank (nusxalash) natijasi
"1 - "9	oxirgi o'chirishlar tarixi
"+	tizim clipboard'i (boshqa dasturlar bilan almashish uchun)
".	oxirgi kiritilgan matn

```
"+yy      → qatorni tizim clipboard'iga nusxalash (masalan brauzerga
joylashtirish uchun)
"+p      → tizim clipboard'idan joylashtirish
```

12. Marks va Jumps

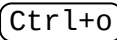
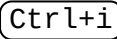
Katta fayllarda tez harakatlanish uchun **marks (belgilar)** ishlatiladi:

```
ma      → joriy joyni "a" belgisi sifatida saqlash
`a      → "a" belgisiga qaytish (aniq pozitsiya)
'a      → "a" belgisi turgan qatorga qaytish
```

Avtomatik marklar ham mavjud:

Mark	Vazifasi
 ``	oxirgi joy (jump qilishdan oldingi pozitsiya)
 `.	oxirgi o'zgartirilgan joy
 `^	Insert rejimdan oxirgi chiqilgan joy

Jump list

Buyruq	Vazifasi
 Ctrl+o	oldingi joyga qaytish (older)
 Ctrl+i	keyingi joyga o'tish (newer)

13. Bir nechta fayl bilan ishlash: Buffers, Windows, Tabs

Buffers — ochilgan fayllar

```
:e fayl2.txt      → yangi faylni ochish
:ls yoki :buffers → ochiq bufferlar ro'yxati
:b2               → 2-raqamli bufferga o'tish
:bn              → keyingi bufferga
:bp              → oldingi bufferga
:bd              → bufferni yopish
```

Windows — ekranni bo'lish (split)

```
:sp fayl.txt      → gorizontal split
:vsp fayl.txt     → vertikal split
Ctrl+w s         → gorizontal split (joriy fayl bilan)
```

```
Ctrl+w v      → vertikal split
Ctrl+w w      → keyingi oynaga o'tish
Ctrl+w h/j/k/l → tegishli yo'nalishdagi oynaga o'tish
Ctrl+w q      → joriy oynani yopish
```

Tabs

```
:tabnew fayl.txt → yangi tab ochish
gt              → keyingi tab
gT             → oldingi tab
:tabclose       → tabni yopish
```

14. Macros (Makroslar)

Macro — bir ketma-ket buyruqlarni yozib olib, keyin qayta-qayta ishlatish imkonini beradi. Bu **takrorlanuvchi ishlarni** avtomatlashtirish uchun juda foydali.

```
qa      → "a" registriga yozishni boshlash (recording)
... (istalgan buyruqlar) ...
q       → yozishni to'xtatish

@a      → "a" makrosini bir marta ishga tushirish
5@a     → "a" makrosini 5 marta ishga tushirish
@@      → oxirgi ishlatilgan makrosni qayta ishga tushirish
```

Misol: Har bir qatorga nuqta-vergul (;) qo'shish kerak bo'lsa:

```
qa      → yozishni boshlash
A;<Esc> → qator oxiriga o'tib, ";" qo'shish
j       → keyingi qatorga tushish
q       → yozishni tugatish
10@a    → shu amalni yana 10 marta takrorlash
```

15. .vimrc sozlash

`~/.vimrc` fayli — Vim'ning shaxsiy sozlamalar fayli. Docker konteynerida ishlaganda, bu faylni konteyner ichiga nusxalash yoki minimal versiyasini qo'lda yozish foydali bo'ladi.

```
vim
```

```

" ===== Asosiy sozlamalar =====
set number                " qator raqamlarini ko'rsatish
set relativenumber        " nisbiy raqamlash (harakatlanish uchun qulay)
set expandtab              " Tab bosilganda probel qo'yish
set tabstop=4             " Tab kengligi
set shiftwidth=4          " indent kengligi
set autoindent            " avtomatik indent
set smartindent           " aqlli indent (kod uchun)
set incsearch             " qidiruv natijasini yozayotganda ko'rsatish
set hlsearch              " qidiruv natijalarini ajratib ko'rsatish
set ignorecase            " qidiruvda katta-kichik harfga e'tibor bermaslik
set smartcase             " lekin katta harf yozilsa, farqlash
set clipboard=unnamedplus " tizim clipboard bilan integratsiya
syntax on                 " sintaksisni ranglash
set encoding=utf-8

" ===== Qulay tugmalar (mappings) =====
let mapleader = " "        " Space tugmasini leader qilish
nnoremap <leader>w :w<CR>    " Space+w → saqlash
nnoremap <leader>q :q<CR>    " Space+q → chiqish
nnoremap <esc> :noh<CR><esc> " qidiruv ajratmasini o'chirish

```

Konteynerda tez sozlash uchun minimal buyruq:

```

bash

echo "set number
set expandtab
set tabstop=4
set shiftwidth=4
set incsearch
set hlsearch" > ~/.vimrc

```

16. Pluginlar

Plugin menejerlaridan eng mashhuri — **vim-plug**.

O'rnatish

```

bash

curl -fLo ~/.vim/autoload/plug.vim --create-dirs \
    https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim

```

`.vimrc` ichida ishlatish

```
vim

call plug#begin('~/.vim/plugged')

Plug 'preservim/nerdtree'           " fayl daraxti (file explorer)
Plug 'vim-airline/vim-airline'     " chiroyli statusbar
Plug 'tpope/vim-fugitive'          " Git integratsiyasi
Plug 'junegunn/fzf.vim'            " tezkor fuzzy fayl qidiruv
Plug 'preservim/nerdcommenter'     " tez kommentariya qo'shish

call plug#end()
```

So'ngra Vim ichida `(:PlugInstall)` buyrug'ini bering.

Eslatma: Docker konteynerlari uchun pluginlar ortiqcha bo'lishi mumkin — konteyner har safar yangi bo'lsa, pluginlarni har safar o'rnatish kerak bo'ladi. Shu sabab, konteynerlar uchun **sof (vanilla) Vim** buyruqlarini yaxshi bilish muhimroq.

17. Docker va server muhitida Vim

Konteynerda Vim yo'q bo'lsa

Ko'p minimal image'larda (`alpine`), (`slim`) versiyalar) Vim oldindan o'rnatilmagan bo'ladi:

```
bash

# Debian/Ubuntu asosidagi image
apt-get update && apt-get install -y vim

# Alpine asosidagi image
apk add vim

# CentOS/RHEL asosidagi image
yum install -y vim
```

Doimiy ishlatiladigan image bo'lsa, buni `(Dockerfile)` ichiga qo'shib qo'yish tavsiya etiladi:

```
dockerfile

RUN apt-get update && apt-get install -y vim && rm -rf /var/lib/apt/lists/*
```

Agar Vim ham o'rnatib bo'lmasa

Ba'zida minimal image'da faqat `(vi)` (Vim emas) mavjud bo'ladi — asosiy `(hjkl)`, `(i)`, `(:wq)`

kabi buyruqlar baribir ishlaydi, chunki ular Vi'dan meros qolgan.

Amaliy maslahatlar

- `docker exec -it <konteyner> vim fayl.txt` — konteyner ichidagi faylni to'g'ridan-to'g'ri tahrirlash.
- Katta o'zgarishlar uchun, faylni host mashinaga nusxalab (`docker cp`), tanish editor'da tahrirlab, qaytarib nusxalash ham mumkin — lekin tez tuzatishlar uchun Vim ancha qulay.
- `crontab -e`, `git commit` (message yozish oynasi), `visudo` — bularning barchasi default holda Vim/Vi'ni chaqiradi, shuning uchun asosiy buyruqlarni bilish shart.
- `docker-compose.yml`, `Dockerfile`, `.env` kabi fayllarni server ustida tez tahrirlash uchun ushbu darslikdagi Normal rejim buyruqlari (`dd`, `yy`, `p`, `:%s///g`) kundalik ishda eng ko'p ishlatiladi.

18. Cheat Sheet

Harakatlanish

<code>h j k l</code>	chap/past/yuqori/o'ng
<code>w b e</code>	so'z oldinga/orqaga/oxiriga
<code>0 ^ \$</code>	qator boshi/birinchi belgi/oxiri
<code>gg G</code>	fayl boshi/oxiri
<code>{n}G</code>	n-qatorga o'tish
<code>Ctrl+d/u</code>	yarim ekran past/yuqori
<code>%</code>	mos qavsga

Tahrirlash

<code>i a o I A O</code>	Insert rejimga kirish (turli usullar)
<code>x X</code>	belgini o'chirish
<code>dd D</code>	qatorni/qator oxirigacha o'chirish
<code>yy Y</code>	qatorni nusxalash
<code>p P</code>	joylashtirish (keyin/oldin)
<code>u Ctrl+r</code>	undo/redo
<code>.</code>	oxirgi buyruqni takrorlash
<code>r <belgi></code>	bitta belgini almashtirish

Operator + Motion

<code>dw cw yw</code>	so'z bo'yicha o'chirish/o'zgartirish/nusxalash
<code>d\$ c\$ y\$</code>	qator oxirigacha
<code>ciw diw yiw</code>	so'zning o'zi bilan (inner word)

```
ci" di" yi" qo'shtirnoq ichida  
ci( di( yi( qavs ichida
```

Visual

```
v V Ctrl+v belgi/qator/blok rejimi
```

Qidirish/Almashtirish

```
/matn ?matn oldinga/orqaga qidirish  
n N keyingi/oldingi moslik  
:%s/a/b/g butun fayldagi "a"ni "b"ga almashtirish
```

Fayl/Buffer

```
:w :q :wq :q! saqlash/chiqish/saqlab chiqish/majburiy chiqish  
:e fayl faylni ochish  
:sp :vsp split (gorizontal/vertikal)  
Ctrl+w + h/j/k/l oynalar orasida o'tish
```

Keyingi qadamlar

1. `(vimtutor)` buyrug'ini terminalda ishga tushiring — Vim ichidagi interaktiv rasmiy darslik (30-40 daqiqa).
2. Bir hafta davomida **faqat Vim'da** kod yozishga harakat qiling — VSCode'ga qaytish istagi kelganda ham chidang.
3. Boshida sichqonchani jismonan uzoqlashtiring — bu majburiy ravishda klaviatura buyruqlarini eslab qolishga yordam beradi.
4. `(.vimrc)` faylingizni asta-sekin, faqat haqiqatan kerak bo'lgan sozlamalar bilan to'ldiring — boshida ko'p plugin o'rnatishga shoshilmang.